جامعة الإمام عبدالرحمن بن فيصل
IMAM ABDULRAHMAN BIN FAISAL UNIVERSITY
كلية علوم الحاسب وتقنية المعلومات
College of Computer Science and Information Technology

CSC 305: Software Engineering
Term: 1
Academic Year: 2025/2026
Course Coordinator: Dr. Rahma Ahmed

FitHub

Smart Gym & Wellness Management System

Software Design Description (SDD)

Supervisor: Ms. Maha Asiri

Table Of Contents

List Of Tables

List Of Figures

1. Introduction

The Software Design Description (SDD) for the FitHub Smart Gym & Wellness Management System provides a comprehensive overview of the system's internal design. It transforms the functional and non-functional requirements into structured architectural, data, and interface specifications. This document ensures that all system components, interactions, and behaviors are well-understood before implementation, reducing development risks and supporting a maintainable, scalable final product.

The SDD serves as a communication bridge between requirement analysis and system development. It clearly defines how the system is organized, how components interact, and how data flows throughout the application.

1.1 Purpose

The purpose of this SDD is to describe the complete design of the FitHub system, including its architecture, data structures, subsystems, and user interface behavior. It provides a detailed blueprint to guide developers and testers during implementation.

This document is intended for:

- **Developers**: to implement system components based on defined architecture and interactions.
- **Testers**: to derive test cases and validate the system's internal behavior.
- **Instructor/Supervisor**: to evaluate the correctness and completeness of the design deliverable.
- **Stakeholders**: to ensure the design aligns with FitHub's objectives and functional needs.

1.2 Scope

This Software Design Description outlines how the FitHub system is structured and how each component contributes to achieving the system's goals. The scope of this document covers the system's architecture, subsystem design, data model, user interface behavior, and communication patterns. It ensures that the system is designed to be efficient, modular, maintainable, and consistent with the requirements described during the analysis phase.

The SDD includes:

- System architecture and design strategy
- Component and module descriptions
- User interface design principles and screen behaviors
- Data entities, schema, and relationships
- Sequence and interaction diagrams
- Communication methods and internal workflows

1.3 Definitions, Acronyms, and Abbreviations

The following table defines the key terms, abbreviations, and acronyms used throughout this Software Design Description to ensure clarity and consistency.

Table 1: Definitions

Acronym	Terminology	Definition
SDD	Software Design Description	Document describing how the system is designed internally.
UI	User Interface	Screens and interactive elements users interact with.
DB	Database	Structured storage system used to maintain application data.
API	Application Programming Interface	Allows communication between software components or services.
Member	System User	A FitHub user who tracks workouts and views plans.
Trainer	Fitness Coach	A user who manages sessions, workouts, and member progress.
Admin	Administrator	Manages users, trainers, system content, and overall configuration.
Workout	Exercise Activity	A physical activity performed by a member and recorded in the system.
Session	Training Session	A scheduled fitness class or appointment led by a trainer.

1.4 References

1. IEEE Recommended Practice for Software Design Descriptions (IEEE 1016).
2. CSC 305: Software Engineering Course Materials, Term 1, AY 2025/2026 — Prepared by Dr. Rahma Ahmed.
3. CSC 305 Software Design Description Template (Adapted for Student Projects).

2. System Overview

FitHub is an online fitness management platform aimed at facilitating organized communication between admins, gym members, and trainers. The platform offers essential features like workout recording, calorie monitoring, progress tracking, membership oversight, appointment scheduling, and performance analytics. It caters to three primary user roles, Admins, trainers, and members each equipped with specialized interfaces and functionalities. The platform prioritizes user-friendliness, data protection, and scalability while ensuring a smooth process for managing fitness and wellness.

2.1 System Functionalities

2.1.1 General Functionalities

- User Authentication: Allows members, trainers, and admins to securely log in and access their respective interfaces.
- Progress and Performance Tracking: Generates weekly and monthly analytics, including charts for workouts, calorie intake, and overall performance.

- Gamification: Awards achievement badges for streaks, workout milestones, and personal goals.

2.1.2 Admin Functionalities

- Create, edit, and disable trainer accounts as needed.
- Manually create member or trainer accounts.
- Activate or deactivate user accounts (e.g., for violating members or trainers who leave the gym).

2.1.3 Trainer Functionalities

- Review active members, session statistics, and recent activity.
- Manage member records.
- Create and update workout sessions.
- View performance analytics and reports for assigned members.
- View and update personal profile and career information.

2.1.4 Member Functionalities

- Review key performance indicators and recent activity summaries.
- Log workouts, review workout history, and access summary statistics that feed streaks and analytics.
- Discover available sessions and manage bookings.
- Track performance trends over weekly, monthly, and overall time periods.
- Monitor streaks and activity breakdowns.
- View and update personal profile information.

2.2 Design Strategy, Architectural Style

The design strategy for FitHub focuses on creating a modular, scalable, and maintainable system that supports three primary user roles, admins, trainers, and members, through a clear separation of concerns. The system follows a structured approach that divides functionality into independent components responsible for user management, workout tracking, scheduling, progress visualization, and analytics.

To ensure long-term flexibility, the system adopts a multi-layered design where the user interface, application rules, and data storage remain isolated from one another. This design makes it easier to update individual components without affecting the entire system. Emphasis is placed on reusability, security, and performance efficiency. Role-based access control ensures that admins, trainers, and members interact only with features relevant to them, while the system remains robust and consistent across different usage scenarios.

Internally, the server follows a three-tier layered architecture, consisting of:

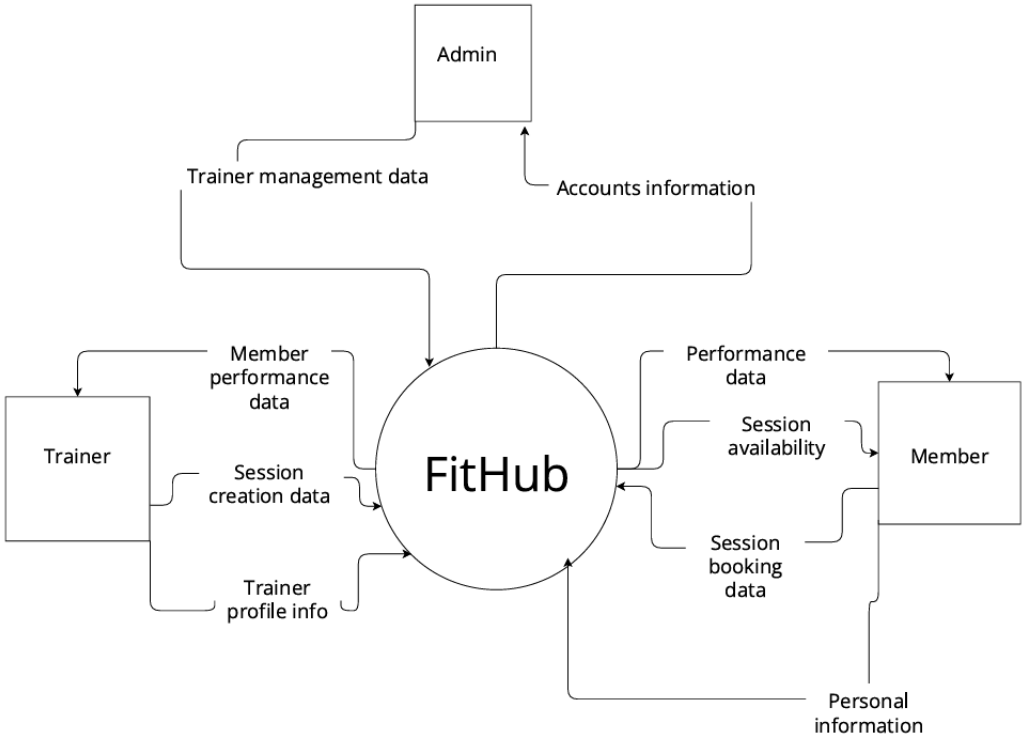
1. Presentation Layer
Handles user interaction through responsive web pages and dashboards. It processes input from users and displays personalized data such as workout logs, progress charts, achievements, and schedules.

2. Application Logic Layer
Implements the system's core operations, including exercise calculations, streak and badge logic, membership management, performance tracking, and communication between users. It enforces the rules that govern fitness programs and trainer–member interactions.

3. Data Layer
Manages persistent storage of user accounts, workout logs, calorie entries, schedule data, achievements, and system configurations. This layer ensures secure and reliable data operations through structured database models.

This architectural combination supports scalability, security, and ease of maintenance, making it well-suited for a fitness environment with ongoing data entry, continuous progress updates, and multiple interacting user roles.
The detailed Architectural Design Diagram (Layered Architecture Diagram) is provided in the **Architectural Design** section.

Context Diagram



3. Design Considerations

The design of the FitHub Smart Gym & Wellness Management System is influenced by several assumptions, constraints, design goals, and trade-offs that shaped the architectural and implementation decisions. These considerations ensure that the system remains scalable, secure, and maintainable while accommodating the needs of both Members and Trainers.

3.1 Assumptions

The design of FitHub is based on the following key assumptions:

1. **Users have access to stable internet and a modern web browser** that supports HTML5, CSS3, and JavaScript.
2. **System traffic is moderate**, primarily serving students, gym members, and trainers within a limited user base.
3. **Users will provide accurate information** when logging workouts, bookings, or updating personal data.
4. **Most operations are lightweight**, such as retrieving session schedules, logging workouts, or viewing dashboards, which do not require heavy computation.
5. **The system is deployed as a web-based application**, with no mobile app in the current phase.

6. **The system includes an Admin role** responsible for managing users, monitoring system activity, and maintaining data integrity
7. **Forgot password requests** are handled through Admin support after user identity verification

3.2 Constraints

The system design is constrained by:

1. **Technology Constraints**
   - The backend must be developed using **Python Flask**, as required by the course project.
   - Data storage must use **SQLite**, which limits concurrent write operations and database size.
2. **Performance Constraints**
   - Response time should remain within **2–3 seconds** under normal load.
   - Heavy operations such as dashboard analytics must be optimized due to SQLite limitations.
3. **Security Constraints**
   - All credentials must be encrypted.
   - Communication must use HTTPS and secure session management.
   - Admin operations require higher-level permissions and must be protected by strict RBAC rules to prevent unauthorized access
4. **Deployment Constraints**
   - FitHub must operate on a lightweight server environment suitable for academic deployment.
   - No cloud services or paid APIs are allowed.
5. **Interface Constraints**
   - The system must follow the UI design and navigation structure provided in the Figma prototype.

3.3 Design Goals

Several goals guided the design and structuring of system components:

1. **Modularity**
   Each feature (authentication, sessions, workouts, attendance) is designed as an independent module to simplify development and testing.
2. **Maintainability**
   The system follows the **MVC architecture** to separate concerns and reduce code complexity.

3. **Scalability**
   Although built for small-scale usage, the architecture supports future migration from SQLite to MySQL/PostgreSQL.
4. **Usability**
   The UI must remain simple and intuitive, especially for non-technical users such as gym members.
5. **Security**
   Strong password hashing, secure sessions, and role-based access control (RBAC) are key priorities.
6. **Administrative Control**
   Provide the Admin with clear and efficient tools to manage users, sessions, and overall system maintenance.

3.4 Design Trade-Offs

During design, several trade-offs were made:

1. **SQLite vs. More Advanced DBMS**
   - SQLite is easier to set up and sufficient for academic scope.
   - But it limits scalability and concurrent writes.
   - Trade-off: **Ease of implementation** vs **performance**.
2. **Pure Flask MVC vs. Large Frameworks (e.g., Django)**
   - Flask offers flexibility and simplicity.
   - Django provides built-in authentication and admin tools.
   - Trade-off: **Lightweight freedom** vs **feature-rich structure**.
3. **Web Application Only vs. Mobile App**
   - Web is faster to develop and meets course requirements.
   - Mobile app would offer better accessibility.
   - Trade-off: **Development effort** vs **multi-platform reach**.
4. **Simple UI vs. Feature-Heavy Dashboard**
   - Simple UI is easier to test and keeps load low.
   - Feature-heavy dashboard could slow the system.
   - Trade-off: **Performance** vs **visual complexity**.

5. **Adding an Admin Role vs. Simplifying System Permissions**
    - The Admin role improves control, allowing for effective monitoring, user management, and system maintenance.
    - However, it introduces additional complexity to the system, requiring stricter RBAC rules, more security checks, and additional UI components.
    - Trade-off: Enhanced administrative control and oversight vs. increased design complexity and security requirements.

4. Architectural Design

4.1 System Architecture Description

This section describes the overall architectural design of the FitHub system, including the main layers, runtime components, and deployment environment. The FitHub system utilizes a **Layered Architecture** pattern combined with the **Model-View-Controller (MVC)** design pattern provided by the Flask framework. This structure ensures a clean separation of concerns, making the system easier to maintain and scale.

The architecture is divided into three primary layers:

1. **Presentation Layer (Client-Side):** This is the user interface accessible via a web browser. It is built using HTML5, CSS3, and JavaScript. It handles user interactions and displays data to Members, Trainers, and Admin.

2. **Application Logic Layer (Server-Side):** This layer acts as the bridge between the user and the data. It is powered by the **Python Flask** framework. It processes HTTP requests, executes business logic (such as validating session bookings or calculating workout streaks), and determines which view or API response to return.

3. **Data Layer (Persistence):** layer is responsible for data storage and retrieval. The system uses SQLAlchemy ORM to communicate with SQLite (a relational database) to persist user profiles, workout logs, session schedules, and attendance records.

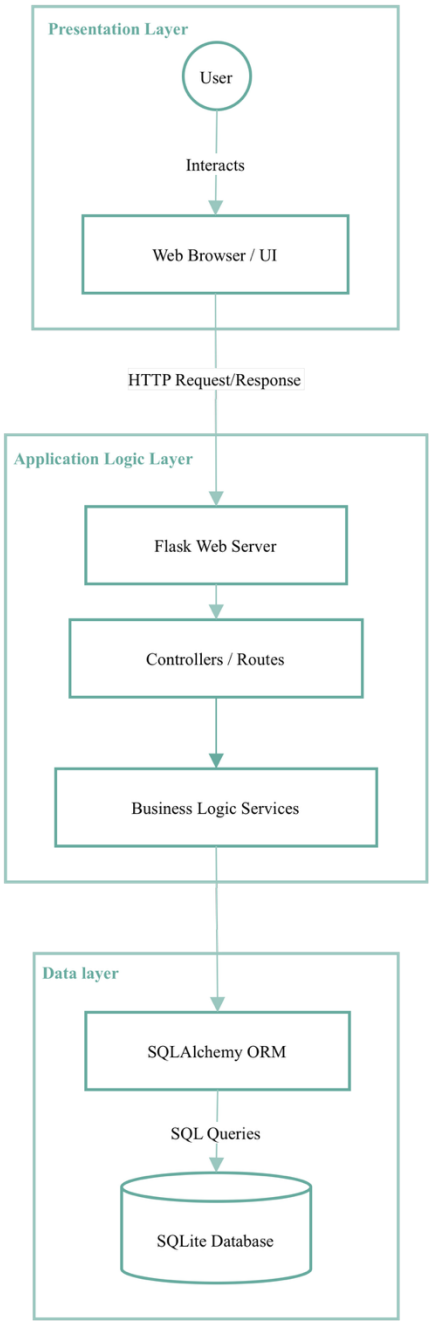


Figure 2: Architectural design diagram

4.2 Component Decomposition

1. **Web User Interface (UI):** Renders HTML templates for Member, Trainer, and Admin dashboards. Captures user inputs (login credentials, workout logs, booking requests). The login page includes a "Forgot Password" option that displays instructions to contact the Admin for assistance. Sends HTTPS commands to the Central Backend Controller.

2. **Central Backend Controller (Flask):** The main entry point for the server. Receives all incoming HTTP requests. Routes specific tasks to the Authentication, Member, Admin or Trainer handlers. Returns rendered views or JSON responses to the UI.

3. **Authentication Handler:** Manages system security. Validates login credentials against stored hashes. Enforces Role-Based Access Control (RBAC) to ensure role isolation between Member, Trainer, and Admin. For password assistance, the system displays contact instructions for the Admin (no automated recovery workflow)

4. **Admin Request Handler:** Provides administrative management features. Allows Admin to create Member/Trainer accounts manually, edit trainer data, and activate/deactivate Member and Trainer accounts. Admin access is granted only to the single pre-configured Admin account (no public registration)

5. **Member Request Handler:** Processes logic for logging workouts and retrieving history. Handles session booking requests by checking availability and capacity limits before confirming.

6. **Trainer Request Handler:** Manages the creation and modification of training sessions. Aggregates attendance data and calculates KPIs to generate performance reports.

7. **Database Interface (ORM):** Acts as the abstraction bridge between the logic handlers and the database. Uses SQLAlchemy to translate high-level Python objects/commands into SQL queries and to map query results back into domain objects.

8. **SQLite Database:** The physical persistent storage. Stores relational tables for users, Workouts, Sessions, Attendance, and Achievements. Supplies data back to the ORM upon query execution.

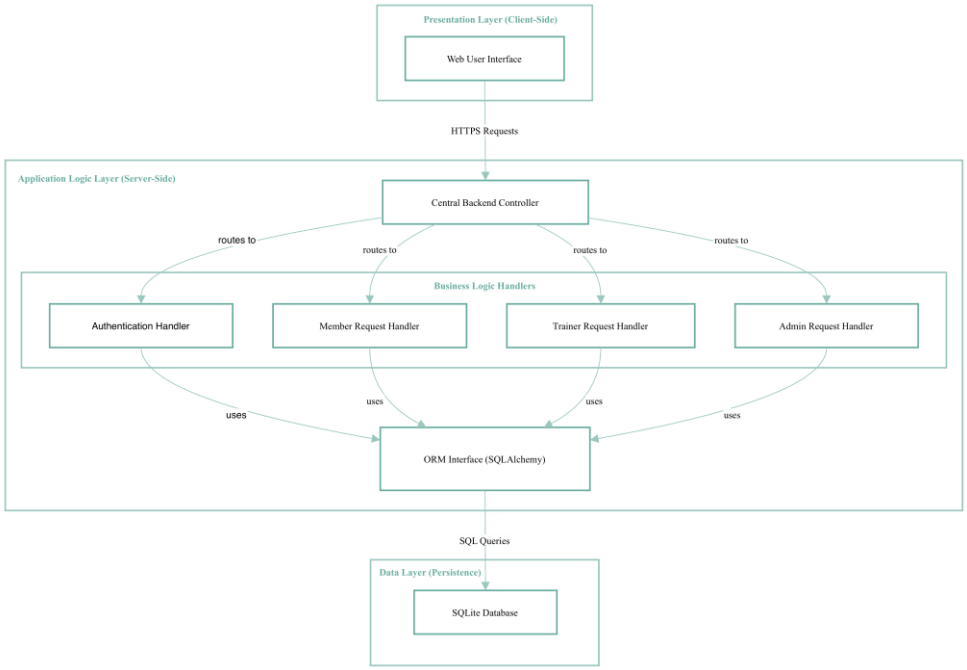


*Figure 3: **component diagram***

Interaction Overview: The Web UI sends HTTPS requests to the Central Backend Controller. The controller routes requests to the appropriate handler (Authentication, Member, Trainer, or Admin). The Authentication Handler validates credentials and enforces RBAC. Domain handlers (Member/Trainer/Admin) execute business rules and use the ORM (SQLAlchemy) to access the SQLite database. The system then returns rendered views or JSON responses back to the UI.

4.3 Deployment Diagram

Table 2: Deployment Diagram

Node Name	Hosted Components	Role and Function
User Workstation (Presentation Tier)	Web Browser (Chrome/Edge/Safari), FitHub Client Interface (HTML5, CSS3, JavaScript), Session Cache, Member/Trainer/Admin Actors.	Provides the entire user interface and experience; handles user input and displays rendered views.
Application **Server (Application & Service Tier)**	Flask Web Server, Backend Controllers (Auth, Member, Trainer, admin), Business Logic	Central Logic. Routes client requests, enforces business rules, processes data, manages system state, and persists/retrieves data through ORM into SQLite.

	Services, SQLAlchemy ORM. SQLite Engine + Persistent DB File (fithub.db).	

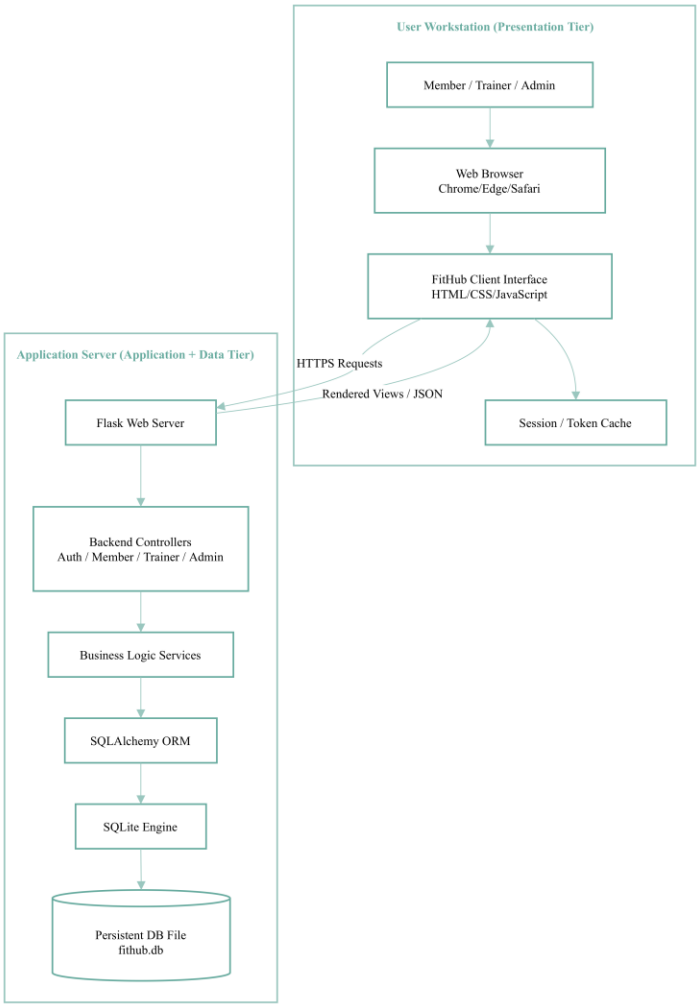


Figure 4: ***Deployment Diagram***

5. Detailed Design

This section describes the detailed design of the FitHub Smart Gym & Wellness Management System. It explains the responsibilities, data, algorithms, inputs/outputs, and interactions of

each main module, and provides UML diagrams (class, sequence, and activity diagrams) that illustrate the system behavior.

5.1 Module Description

Table 5.1 summarizes the main modules of FitHub, their responsibilities, inputs/outputs, and the diagrams that describe their behavior.

Module Name	Responsibilities (Data, Algorithms, Interactions)	Inputs / Outputs	Notes / Diagrams
Authentication & User Accounts Module	Data: - User records stored in the Members, Trainers, and Admin tables tables (user_Id, fullName, email, passwordHash, role, status, createdAt). - Distinguishes between Member, Trainer, and Admin roles using the role attribute. Algorithms: - Validate login and registration form fields (non-empty, valid email format). - Hash passwords before storing them in the database. - Compare the input password with the stored password hash. Decide target dashboard based on user role: – Member → Member Dashboard – Trainer → Trainer Dashboard – Admin → Admin Dashboard (via the Admin Portal). Interactions: - Interacts with Member/Trainer login and registration pages. - Collaborates with authentication services and the user repository to verify credentials. - Starts and terminates user sessions during login and logout.	Inputs: - Login: email, password. The role is determined from the stored user record (Member / Trainer / Admin). - Registration: full name, email, password and confirmation, role. Outputs: - Authentication success or failure. - Error messages for invalid credentials or existing email. - New user account record. - Active session token and redirection to the appropriate dashboard.	Implemented mainly in the authentication controller and service. Works with the Users table to store and verify credentials and manage sessions. Related UML diagrams: - Fig 5.3.1 FitHub Core Entities Class Diagram. - Fig 5.3.3 Member / Trainer Login Sequence Diagram. - Fig 5.3.4 Member Registration Sequence Diagram. - Fig 5.3.5 Logout Sequence Diagram. - Fig 5.3.12 Admin Portal Login Sequence Diagram - Fig 5.3.13 Admin Creates Trainer Account Sequence Diagram

Workout Tracking & Gamification Module	Data: - Workout table (workoutId, memberId, date, type, duration, calories). - Achievement table (achievementId, memberId, name, condition, dateUnlocked). Algorithms: - Calculate calories if not provided, based on workout type and duration. - Update daily and monthly totals for workouts and calories. - Compute and update streak values (for example, consecutive active days). - Evaluate achievement rules (for example, 7-day streak, 50 workouts) and unlock new badges. Interactions: - Used by the My Workouts and Progress & Analytics pages. - Receives workout logs from the member interface. - Notifies analytics and achievement services after saving workouts.	Inputs: - Workout form data: type, date, duration, optional calories. - Member identifier and existing workout history. Outputs: - New workout record stored in the database. - Updated totals (number of workouts, calories, average duration). - Updated streak and achievement status for the member.	Used by the My Workouts and Progress & Analytics pages to store workouts and update member statistics and achievements. Related UML diagrams: - Fig 5.3.1 FitHub Core Entities Class Diagram. - Fig 5.3.6 Log Workout Sequence Diagram. - Fig 5.3.7 View Progress & Analytics Sequence Diagram.
Session Booking & Management Module	Data: - Session table (session_Id, trainer_Id, title, dateTime, duration, capacity, location). - Attendance table (attendance_Id, session_Id, member_Id, status). Algorithms: - Check session capacity before confirming a booking. - Prevent duplicate bookings for the same member and session. - Update booked count and compute fill rate (booked / capacity). - Handle booking cancellation and restore available capacity.	Inputs: - For members: selected session_Id, member_Id, booking/cancellation requests. - For trainers: new or updated session details (title, date and time, duration, capacity, location). Outputs: - New or updated session records. - Booking records (created or cancelled). - Updated capacity, booked count, and fill rate.	Supports both member and trainer flows for managing training sessions and bookings. It coordinates with the sessions and bookings tables and the notification service. Related UML diagrams: - Fig 5.3.1 FitHub Core Entities Class Diagram.

	Interactions: - Used by the Training Sessions page (member portal) and Session Management page (trainer portal). - Interacts with notification services to send booking confirmations. - Communicates with the sessions repository to create, update, and retrieve sessions and bookings.	- Error messages when a session is full or already booked.	- Fig 5.3.2 Activity Diagram – Book Training Session. - Fig 5.3.8 Book Training Session Sequence Diagram. - Fig 5.3.9 Create New Session (Trainer) Sequence Diagram.
Analytics, Reporting & Member Management Module	Data: - Aggregated workout statistics (per week, month, and overall). - Attendance and performance metrics for sessions (attendance rate, fill rate). - Member summaries (attendance percentage, last check-in, total sessions attended). Algorithms: - Aggregate workouts by period (This Week, This Month, Overall). - Compute KPIs such as total workouts, total calories, average session duration, streak length. - Calculate trainer performance metrics (sessions conducted, attendance, fill rate, average rating). - Generate member-level statistics for trainers (attendance percentage, last visit, progress trends). Interactions: - Provides data to the member Progress & Analytics page. - Supplies reports for the trainer Performance Reports page. - Supports the trainer Member Management page by combining profile and analytics data. - provides the high-level KPIs and summaries displayed on the Admin Dashboard (total	Inputs: - Stored workouts, sessions, bookings, and achievements. - Selected time filter (week, month, overall) or selected member/trainer. Outputs: - KPIs and charts for members and trainers. - Detailed performance reports and daily breakdown tables. - Member lists with attendance summaries and status.	Provides aggregated statistics and reports for members and trainers, and supports the trainer member-management view by combining profile and analytics data. Related UML diagrams: - Fig 5.3.1 FitHub Core Entities Class Diagram. - Fig 5.3.7 View Progress & Analytics Sequence Diagram. - Fig 5.3.10 View Trainer Performance Reports Sequence Diagram. - Fig 5.3.11 Member Management (Trainer) Sequence Diagram. - Fig 5.3.14 Admin Updates Member Subscription Sequence Diagram

	trainers, total members, active memberships, revenue).		

Table 3: 5.1Module Description for FitHub.

5.2 Constraints and Composition

Table 5.2 describes the main constraints and composition relations for the FitHub modules. Each component lists its pre-conditions, post-conditions, and key dependencies.

Component	Pre-condition	Post-condition	Dependencies
Authentication & User Accounts Module	The user opens the login or registration page (member, trainer, or admin) and submits the required fields. The database connection and authentication services are available.	For valid data, a member, trainer, or admin account is authenticated, and a session is started. The user is redirected to the corresponding dashboard (Member, Trainer, or Admin).For invalid data, an error message is returned and no session is created.	User database (Members and Trainers tables), password hashing library, session management mechanism (cookies or tokens), email validation utilities.
Workout Tracking & Gamification Module	The member is authenticated and has access to the My Workouts page. Required workout fields (type, date, duration) are provided.	A new workout entry is stored, aggregates (totals, streaks, calories) are updated, and any new achievements are recorded for the member. The updated history and statistics are visible in the UI.	Workout repository, analytics service, achievement rules engine, date and time utilities.
Session Booking & Management Module	The member or trainer is authenticated. For booking, a valid session exists with remaining capacity. For session creation or update, the trainer has permission to manage sessions.	For members: a booking or cancellation is stored, and session capacity is updated. For trainers: a new session is created, or an existing one is updated while keeping bookings consistent.	Sessions repository, user database, notification service (for booking confirmations), calendar utilities.
Analytics, Reporting & Member Management Module	Historical workout and session data exist in the database. The user (member or trainer) is authenticated	Computed statistics and detailed reports are generated and displayed to the user as charts and tables. For trainers, member lists and	Workout and sessions repositories, analytics repository (aggregated

	and selects a valid reporting period or member.	details are updated with the latest analytics.	statistics), member profiles, chart components in the UI.

Table 4: 5.2 Constraints and Composition for FitHub Components.

5.3 UML Diagrams

This subsection presents the UML diagrams used to describe the detailed behavior of FitHub. It includes a core class diagram, an activity diagram for a key flow, and a set of sequence diagrams for the main use cases for **members, trainers, and the admin**.

5.3.1 Class Diagram - Core Entities

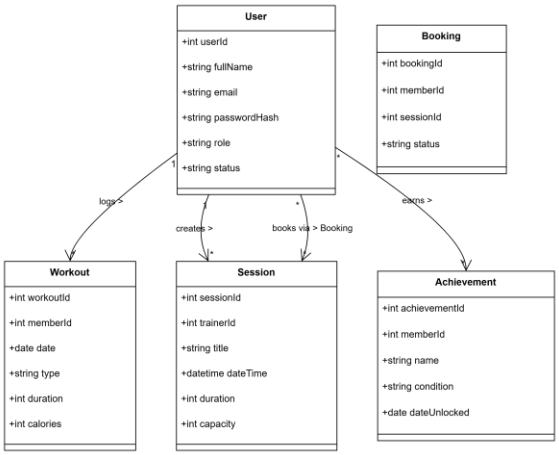


Figure 5: 5.3.1 FitHub Core Entities Class Diagram.

5.3.2 Activity Diagram - Book Training Session

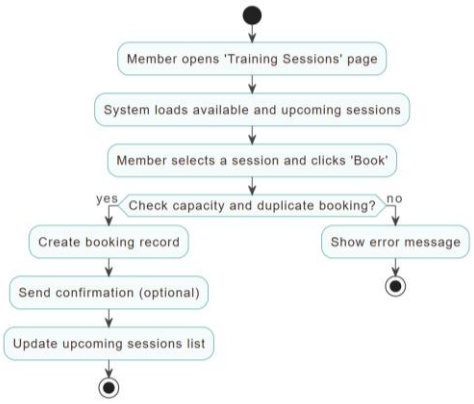


Figure 6: 5.3.2 Activity Diagram for Booking a Training Session.

5.3.3 Member / Trainer Login Sequence Diagram

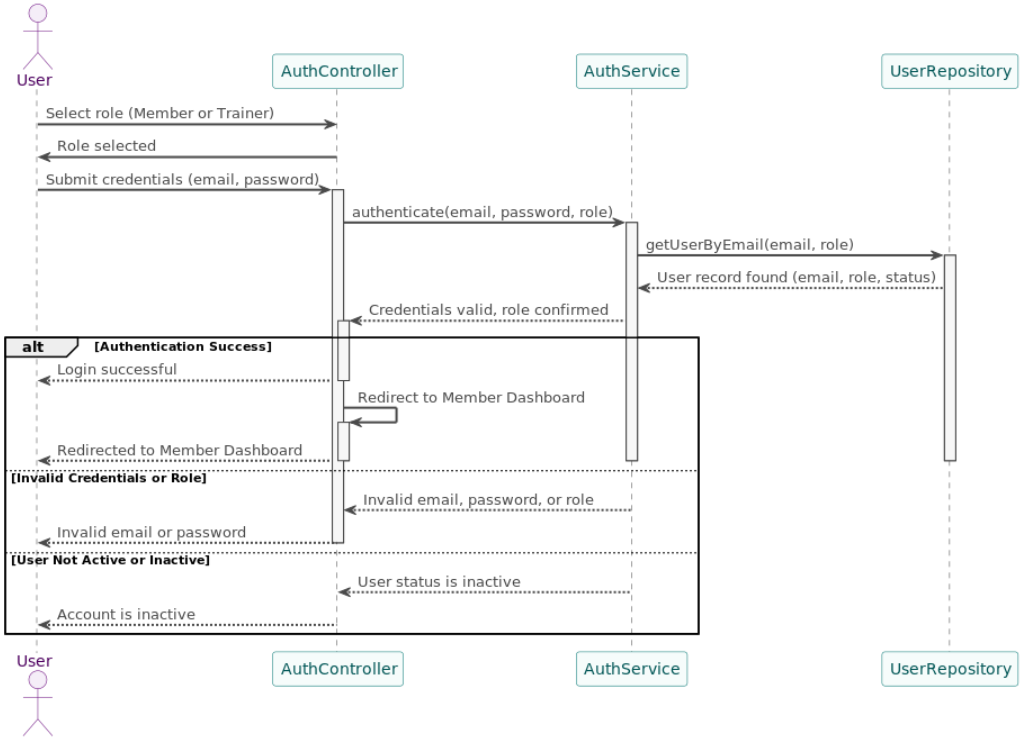


Figure 7: 5.3.3 Member / Trainer Login Sequence Diagram.

5.3.4 Member Registration Sequence Diagram

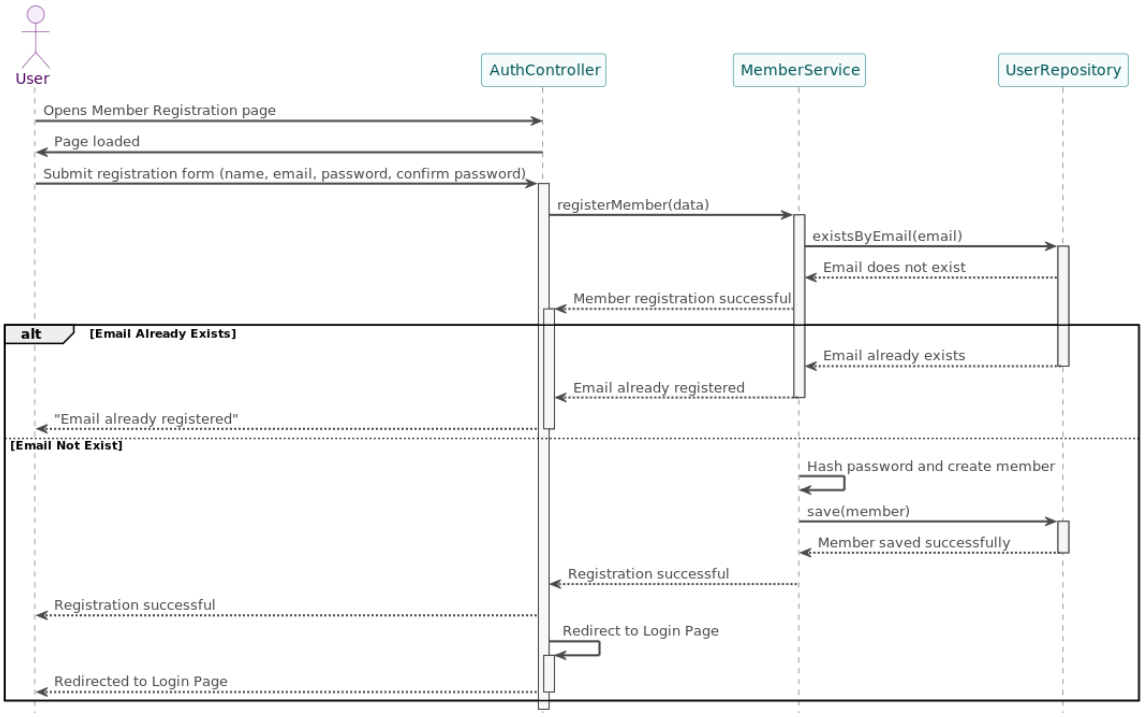


Figure 8: 5.3.4 Member Registration Sequence Diagram.

5.3.5 Logout Sequence Diagram

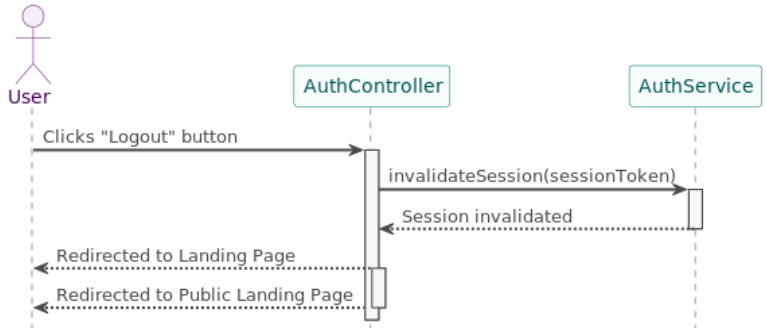


Figure 9: 5.3.5 Logout Sequence Diagram.

5.3.6 Log Workout Sequence Diagram

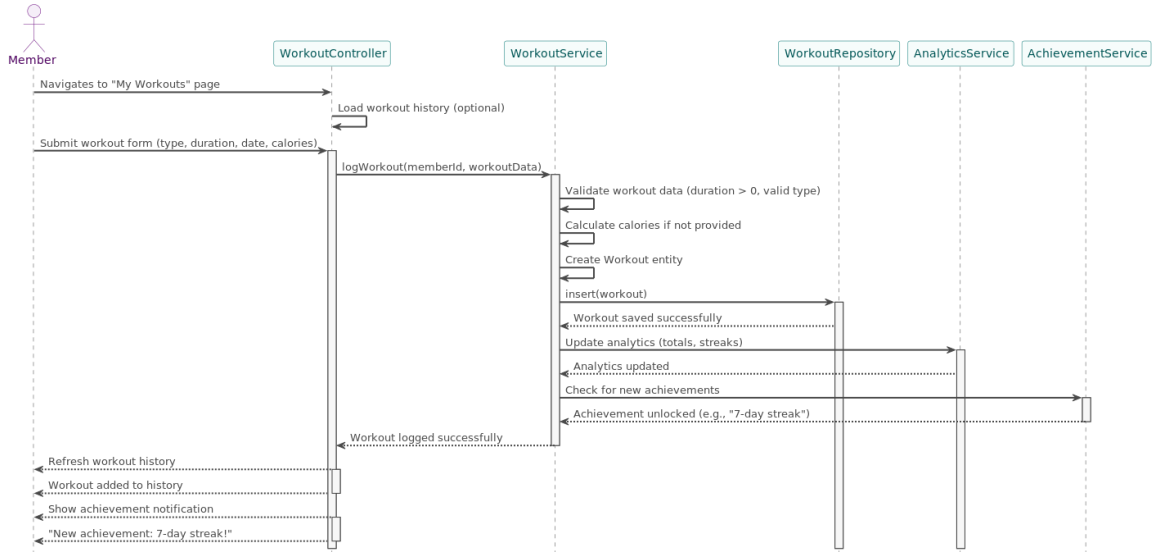


Figure 10: 5.3.6 Log Workout Sequence Diagram.

5.3.7 View Progress & Analytics Sequence Diagram

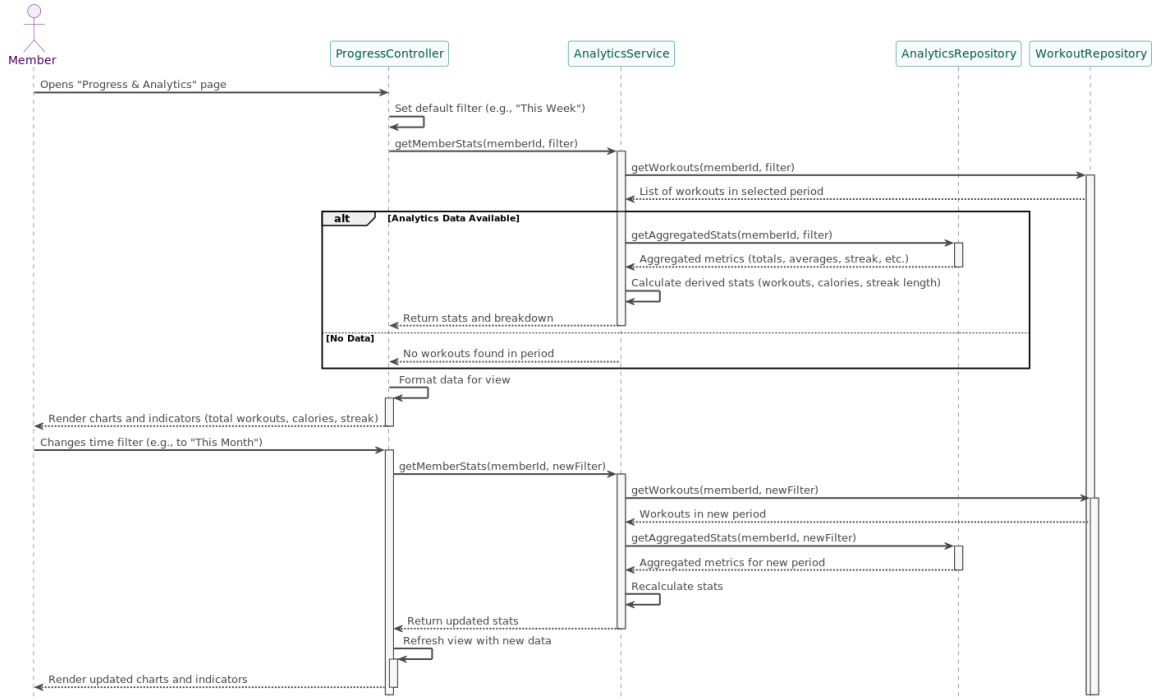


Figure 11: 5.3.7 View Progress & Analytics Sequence Diagram.

5.3.8 Book Training Session Sequence Diagram

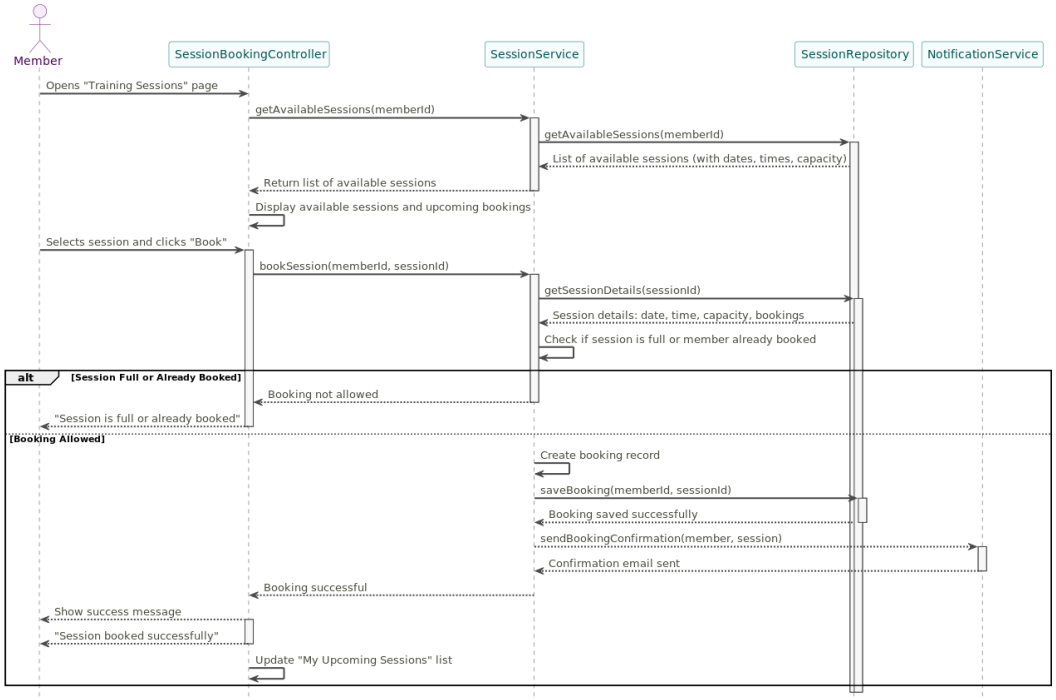


Figure 12: 5.3.8 Book Training Session Sequence Diagram.

5.3.9 Create New Session (Trainer) Sequence Diagram

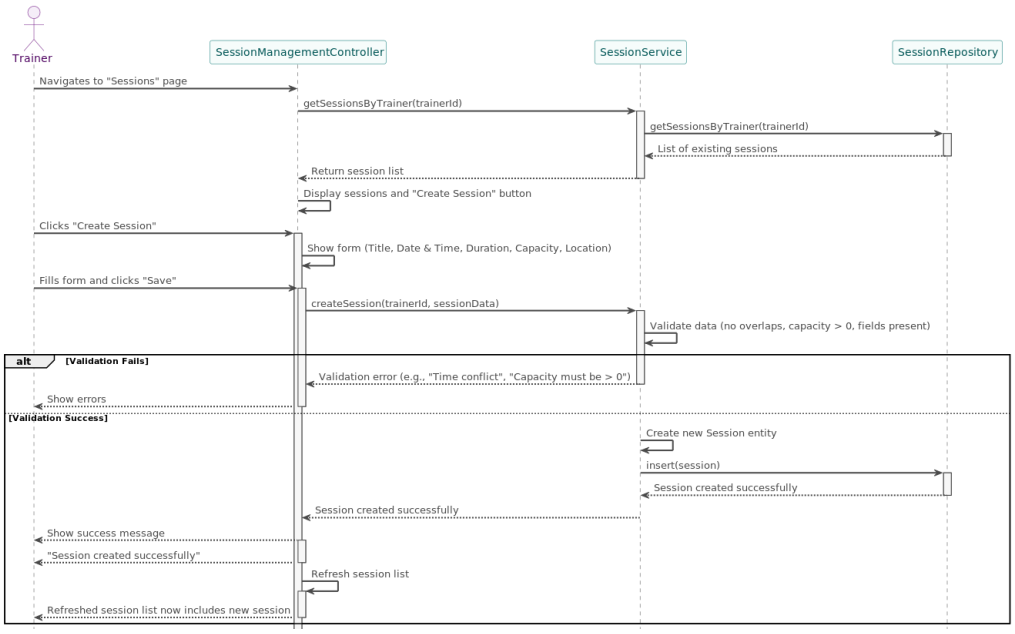


Figure 13: 5.3.9 Create New Session (Trainer) Sequence Diagram.

5.3.10 View Trainer Performance Reports Sequence Diagram

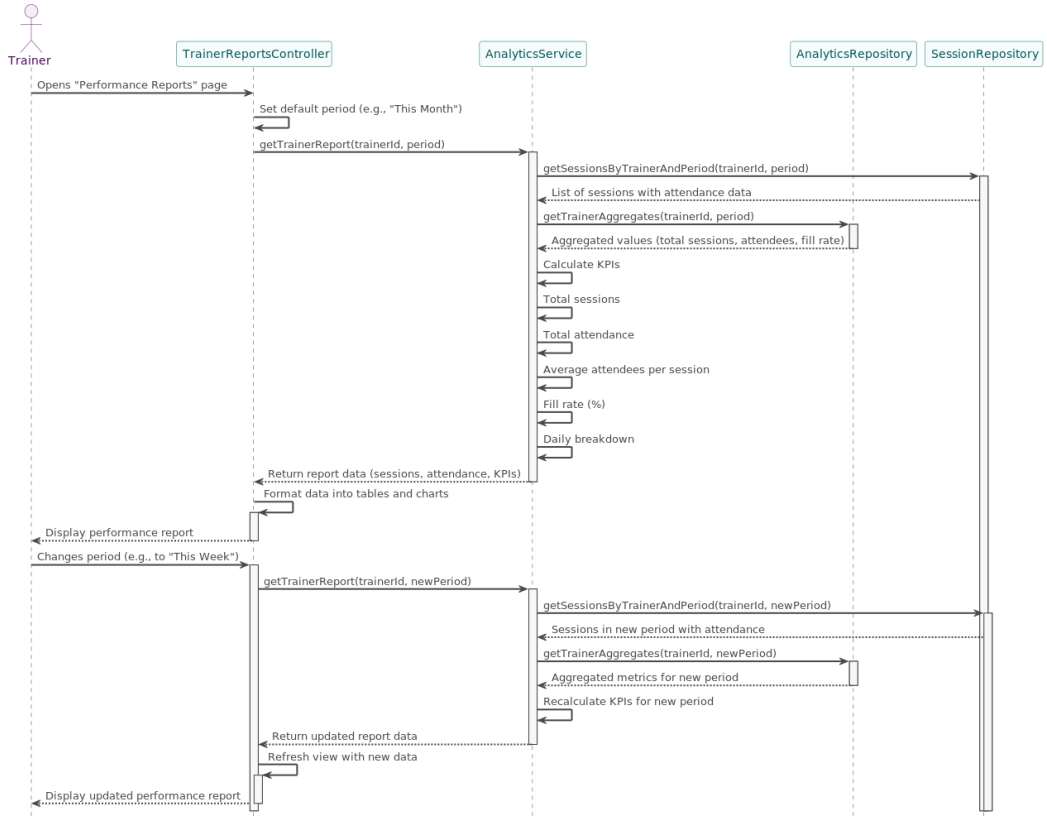


Figure 14: 5.3.10 View Trainer Performance Reports Sequence Diagram.

5.3.11 Member Management (Trainer) Sequence Diagram

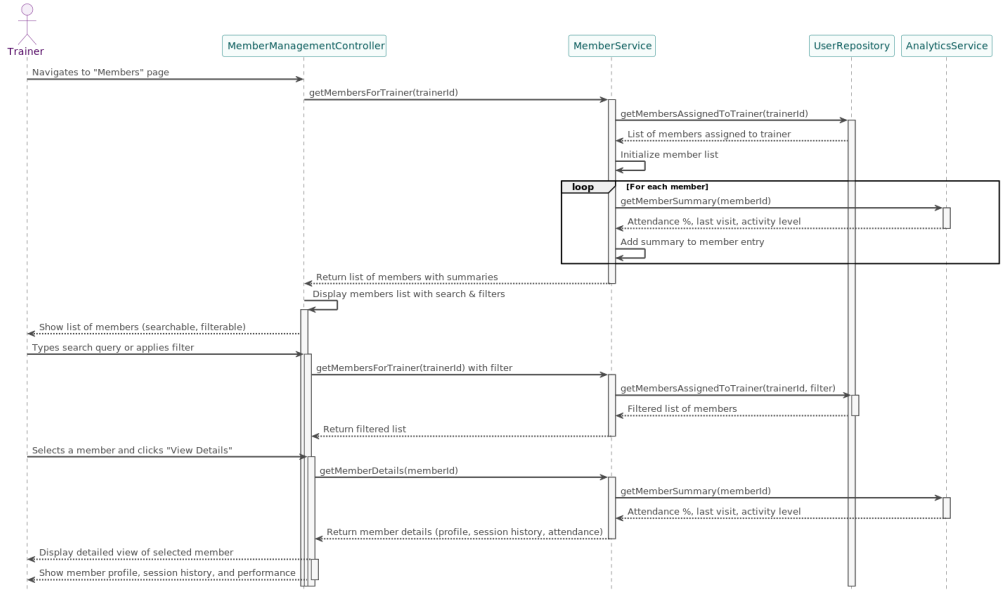


Figure 15: 5.3.11 Member Management (Trainer) Sequence Diagram.

5.3.12 Admin Portal Login Sequence Diagram

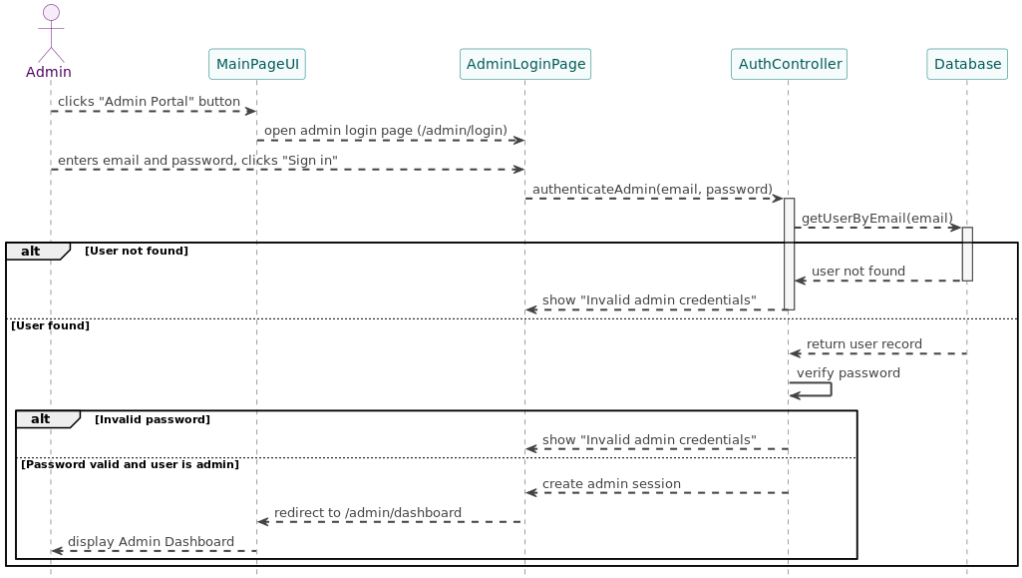


Figure 16: 5.3.12 Admin Portal Login Sequence Diagram

5.3.13 Admin Creates Trainer Account Sequence Diagram

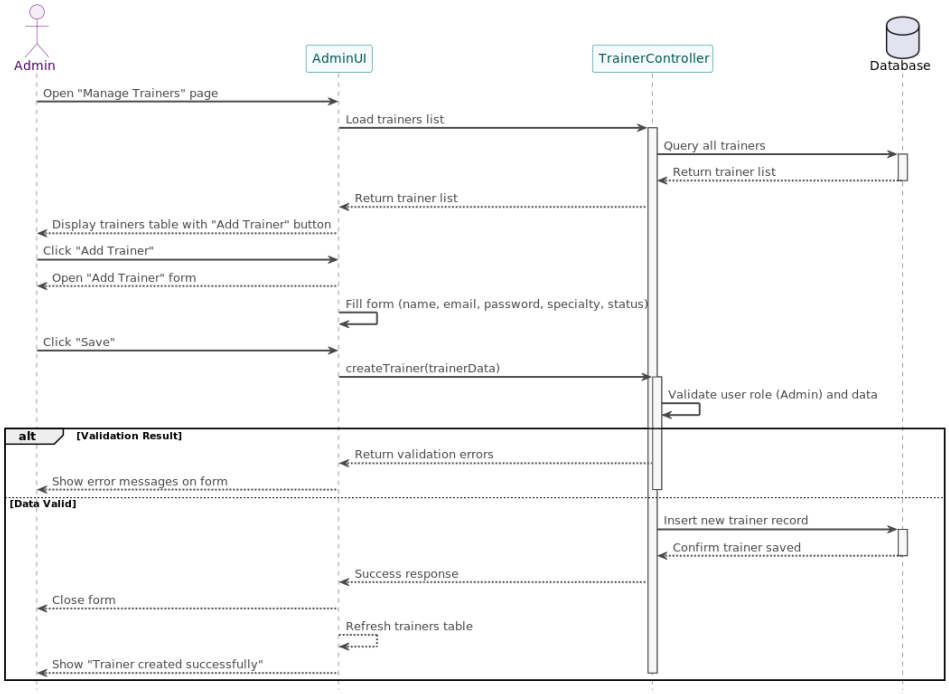


Figure 17: 5.3.13 Admin Creates Trainer Account Sequence Diagram

5.3.14 Admin Updates Member Subscription Sequence Diagram

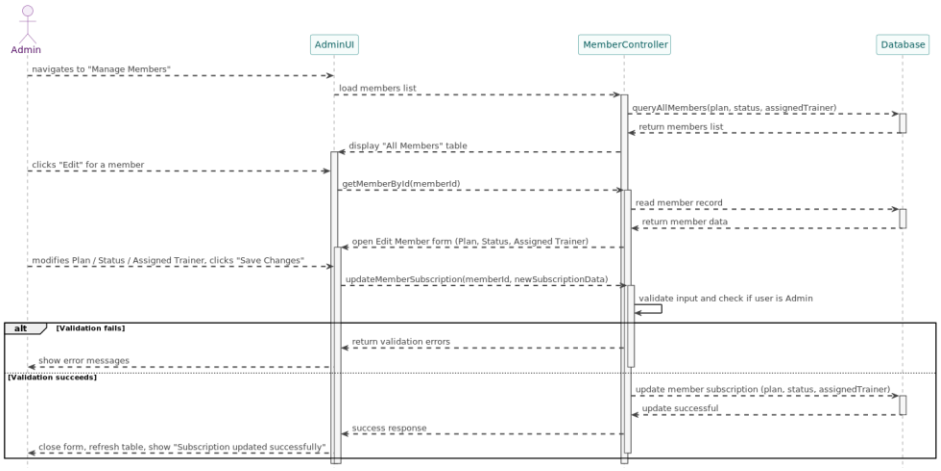


Figure 18: 5.3.14 Admin Updates Member Subscription Sequence Diagram

6. Data Design

This section describes the data design for FitHub. It defines the main data structures, their keys, and the relationships between them. It also presents a textual ER diagram based on the implemented SQL schema.

6.1 Data Description

This subsection provides FitHub database entities, their fields including the data type and constraint for each field.

Table 5: Data Description

Entity	Field	Type	Constraint
Admin	admin_id	INTEGER	PRIMARY KEY AUTOINCREMENT
	name	TEXT	NOT NULL, UNIQUE
	email	TEXT	NOT NULL
	password	TEXT	NOT NULL
Member	member_id	INTEGER	PRIMARY KEY AUTOINCREMENT
	name	TEXT	NOT NULL
	email	TEXT	NOT NULL, UNIQE
	phone_number	TEXT	-
	password	TEXT	NOT NULL
	join_month	TEXT	-
	status	TEXT	DEFUALT 'Active'
	total_points	INTEGER	DEFUALT 0
	level	TEXT	DEFUALT 'Beginner'
	achievements_count	INTEGER	DEFUALT 0
Trainer	trainer_id	INTEGER	PRIMARY KEY AUTOINCREMENT
	name	TEXT	NOT NULL
	email	TEXT	NOT NULL, UNIQE
	password	TEXT	NOT NULL
	phone_number	TEXT	-

	specialty	TEXT	-
	join_month	TEXT	-
	status	TEXT	DEFUALT 'Active'
Session	session_id	INTEGER	PRIMARY KEY AUTOINCREMENT
	trainer_id	INTEGER	NOT NULL, FOREIGN KEY → Trainer(trainer_id)
	session_name	TEXT	NOT NULL
	date	TEXT	-
	time	TEXT	-
	location	TEXT	-
	status	TEXT	DEFUALT 'Scheduled'
Attendance	attendance_id	INTEGER	PRIMARY KEY AUTOINCREMENT
	member_id	INTEGER	NOT NULL, FOREIGN KEY → Member(member_id)
	session_id	INTEGER	NOT NULL, FOREIGN KEY → Session(session_id)
	trainer_id	INTEGER	NOT NULL, FOREIGN KEY → Trainer(trainer_id)
	attendance_date	TEXT	-
	status	TEXT	DEFAULT 'Present'
	points_earned	INTEGER	DEFAULT 0
Workout	workout_id	INTEGER	PRIMARY KEY AUTOINCREMENT
	member_id	INTEGER	NOT NULL, FOREIGN KEY → Member(member_id)
	trainer_id	INTEGER	FOREIGN KEY → Trainer(trainer_id)
	activity_type	TEXT	NOT NULL
	duration	INTEGER	-
	calories	INTEGER	-
	date	TEXT	-
	status	TEXT	DEFAULT 'Completed'

	streak_days	INTEGER	DEFAULT 0
Achievement	achievement_id	INTEGER	PRIMARY KEY AUTOINCREMENT
	member_id	INTEGER	NOT NULL, FOREIGN KEY → Member(member_id)
	title	TEXT	NOT NULL
	description	TEXT	-
	achieved_date	TEXT	-
	badge_icon	TEXT	-

6.2 Data Dictionary

This section Describe all the entities and their fields.

Table 6: Data Dictionary

Entity	Field	Description
Admin	admin_id	Unique identifier for each system administrator, automatically generated by the system
	name	Admin full name
	email	Admin email (used for login) Must be unique
	password	Login password (encrypted)
Member	member_id	Unique identifier for each member
	name	Member's full name
	email	Member's email (used for login) Must be unique
	phone_number	Contact number of member
	password	Login password (encrypted)
	join_month	Month the member joined the gym
	status	Shows if member account is Active/ Inactive
	total_points	Total points collected from workouts and attendance
	level	Member's current fitness level (Beginner, Intermediate, Advanced)

	achievements_count	Number of achievements earned
Trainer	trainer_id	Unique identifier for each trainer
	name	Trainer's name
	email	Trainer's email (used for login)
	password	Login password (encrypted)
	phone_number	Trainer's contact number
	specialty	Type of classes or Skills the trainer teaches
	join_month	Month the trainer joined
	status	Trainer's current employment status
Session	session_id	Unique identifier for each session
	trainer_id	ID of the trainer who conducts the session
	session_name	Title or name of the session
	date	Date of the session
	time	Time of session
	location	Location of the session
	status	Indicates if the session is Scheduled or Completed
Attendance	attendance_id	Unique record for each attendance log
	member_id	Member who attended the session
	session_id	Session that was attended
	trainer_id	Trainer who conducted the session
	attendance_date	Date the member attended the session
	status	Whether the member was Present or Absent
	points_earned	Points received for attendance
Workout	workout_id	Unique identifier for each workout
	member_id	Member who performed the workout
	trainer_id	Trainer supervising the workout (optional)
	activity_type	Type of activity (e.g., cardio, strength)
	duration	Duration in minutes

	calories	Calories burned
	date	Workout date
	status	Workout status (Completed, Missed)
	streak_days	Number of consecutive workout days
Achievement	achievement_id	Unique identifier for each badge or milestone
	member_id	Member who earned the achievement
	title	Achievement title (e.g., 30-day streak)
	description	Description of the achievement
	achieved_date	Date when it was earned
	badge_icon	Path of URL of the badge icon image

6.3 Database Description

The FitHub database is implemented using SQLite, a lightweight relational database engine suitable for web applications. The database schema follows a normalized design to reduce redundancy.

Foreign keys maintain referential integrity between entities, and default constraints ensure that values such as status and total points always have valid defaults.

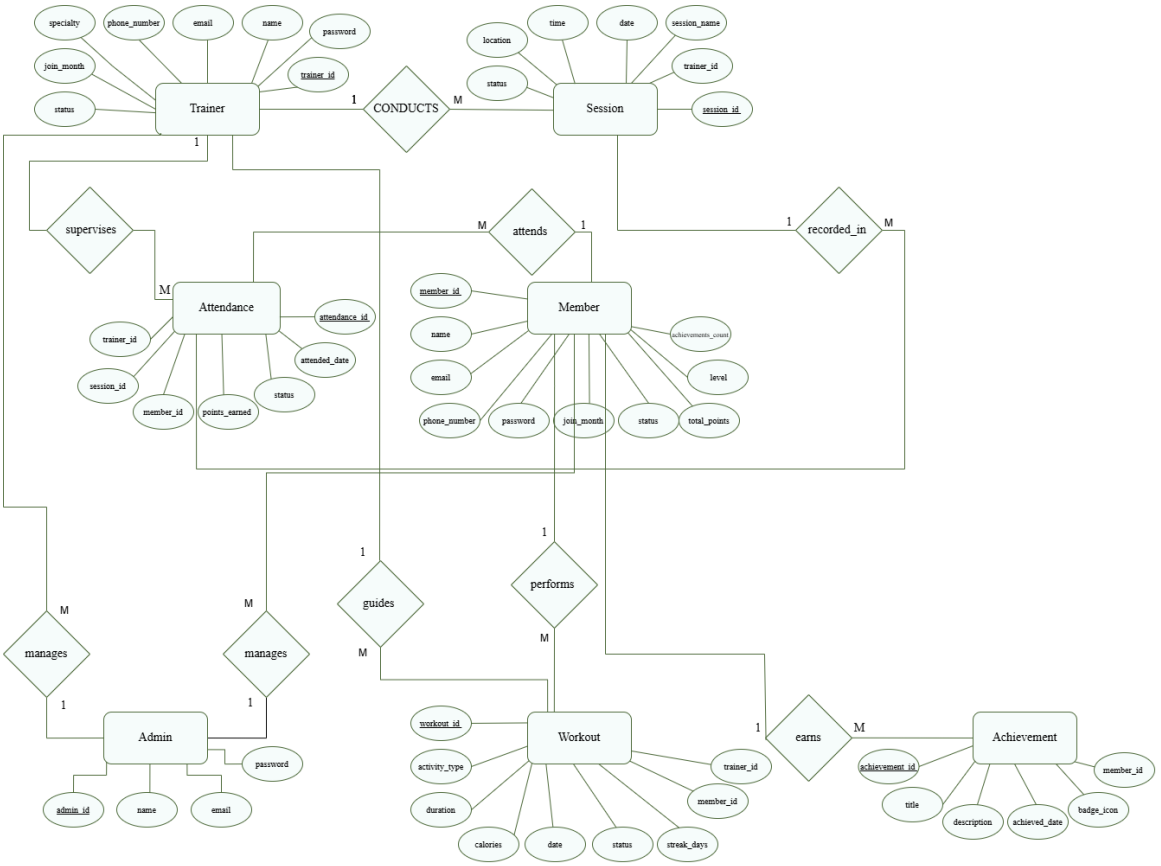


Figure 19: ER Diagram

We shall depict the Relational Mapping in the figure below once it has been converted from the ER format. Following that, we may create the tables in the database.

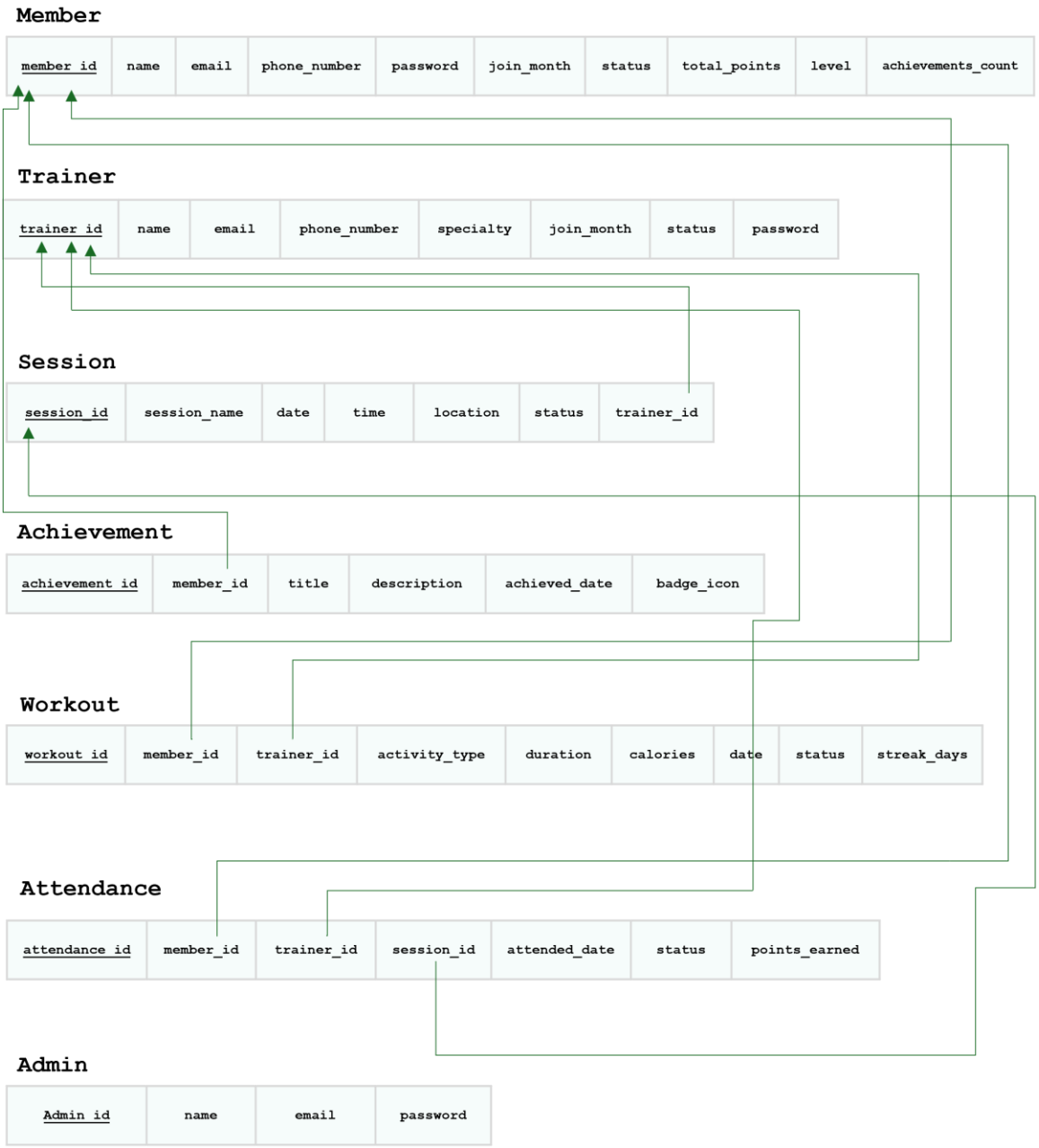


Figure 20: ER Mapping

7. External Interfaces

This section describes how the FitHub system interacts with any external components, technologies, services, or protocols beyond its internal modules. Although FitHub is a standalone web-based application with no third-party integrations, it relies on several external interfaces to function correctly, including web browsers, communication protocols, backend services, databases, and email systems. These interfaces ensure smooth communication between the frontend, backend, and data storage layers of the system.

7.1 Hardware Interfaces

The FitHub system is a fully web-based software application and does not require any specialized hardware for operation. All system functionality is accessible through standard computing devices that support a modern web browser and internet connection.

The system is accessed through:

- Standard laptops or desktop computers
- Tablets or smartphones
- Any modern device capable of running a web browser

FitHub operates fully software-based.

7.2 Software Interfaces

This section describes all external software components, libraries, frameworks, and services that FitHub relies on to function correctly.

7.2.1 Web browser Interface

The browser handles all user interactions such as logging a workout, booking a session, viewing analytics, and navigating dashboards. It communicates with the backend using HTTPS requests and renders any returned JSON-based content dynamically.

This table summarizes the browsers and technologies FitHub depends on.

Table 7: Browsers and Technology

Component	Description
Google Chrome	Fully supported, optimized performance.
Safari	Supported for Apple devices (iPhone, iPad, Mac).
Mozilla Firefox	Supported with standard compatibility.
Microsoft Edge	Supported for Windows devices.
HTML5	Used for page structure and layout rendering.
CSS3	Provides styling, layout, and responsive UI.
JavaScript	Enables dynamic functions such as dashboard updates, form validation, and interactive widgets.

7.2.2 Backend Framework Interface

The backend is powered by a Python Flask web framework, which handles:

- Routing and URL endpoint mapping
- Processing GET/POST/PUT/DELETE requests
- Authentication and session management
- Rendering dynamic HTML templates
- Serving JSON data for analytics and dashboards

Table 8: Backend Responsibilities

Responsibility	Description
Routing	Maps URLs to backend functions.
Request Handling	Parses incoming HTTP requests.
Business Logic Execution	Validates sessions, logs workouts, detect conflicts, etc.
Template Rendering	Builds dynamic pages for members and trainers.
Session Management	Maintains secure user sessions.

7.2.3 Database Interface

The database interface allows FitHub to:

- Save and retrieve user login information
- Store session schedules
- Log workout details and calories
- Track attendance
- Calculate analytics (streaks, totals, reports)
- Store achievements earned by members

The ORM layer automatically converts Python commands into SQL statements, ensuring consistency and reducing errors.

7.2.4 Restful API Interface

FitHub uses internal REST-style communication between the frontend and backend.

Table 9: REST API Methods

Method	Purpose
GET	Retrieve workouts, sessions, profiles, analytics.
POST	Submit forms (login, booking, adding workout).
PUT	Update user or session data.
DELETE	Remove entries (cancel booking, delete workout).

Data Format: JSON
Standard for all server responses. Used for:

- Dashboards
- Analytics charts
- Booking confirmations
- Workout summaries

7.3 Communication Interfaces

This describes the protocols FitHub uses to ensure secure communication.

7.3.1 Network Protocols

FitHub uses:

- HTTPS — encrypts all traffic to prevent interception
- TLS/SSL Certificates — ensures server identity verification
- HTTP/1.1 — request–response model for browser ↔ server communication

These ensure secure data transfer for login, workout logs, bookings, and analytics.

7.3.2 Data Exchange Format

FitHub uses JSON for all communication:

- Lightweight and fast
- Human-readable
- Easily parsed by JavaScript
- Ideal for dashboards that update dynamically

JSON is used in:

- Workout logs
- Session booking
- Analytics graphs
- Profile updates

8. Appendices

Glossary of terms

Term	Definition
Encryption	the process of transforming readable plain text into unreadable ciphertext to mask sensitive information from unauthorized users.

JavaScript Object Notation (JSON)	A simple way to store and send information using text
key performance indicator (KPI)	a quantifiable measure of performance over time for a specific objective.
Module	an extension to a main program dedicated to a specific function.
Model–view–controller (MVC)	a software architectural pattern commonly used for developing user interfaces that divide the related program logic into three interconnected elements.
Object Relational Mapping (ORM)	a technique used in creating a "bridge" between object-oriented programs and, in most cases, relational databases.
Prototype	Prototype: a working model that demonstrates the functionality of an application at an early stage of development.
Role-Based Access Control (RBAC)	a model for authorizing end-user access to systems, applications and data based on a user's predefined role.
Response time	the time taken to transmit the inquiry, process it by the computer, and transmit the response back to the terminal.
Structured query language (SQL)	a standard language for database creation and manipulation.
Uniform Resource Locator (URL)	URL (Uniform Resource Locator): is the address of a unique resource on the internet

ERD Diagram

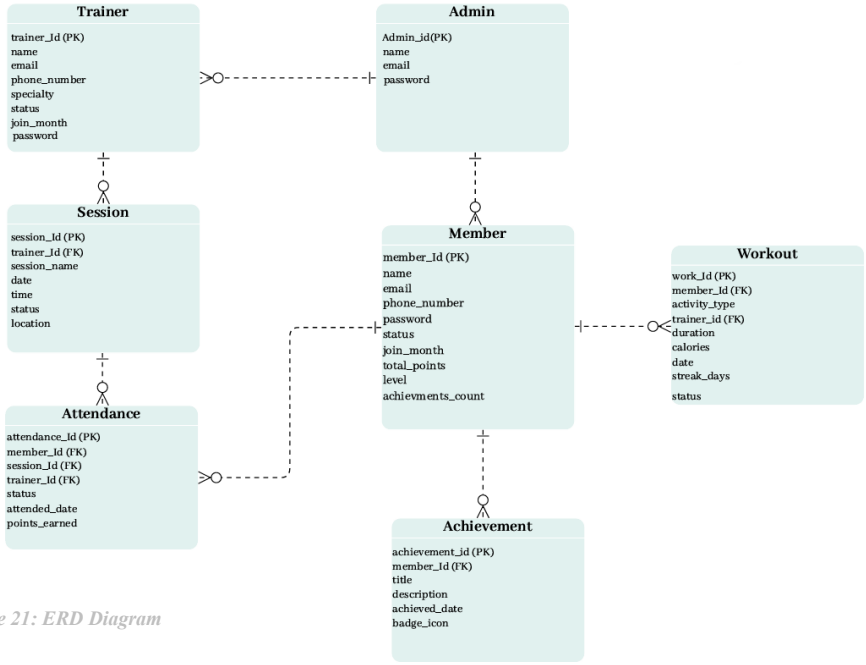


Figure 21: ERD Diagram

Use Case Diagram

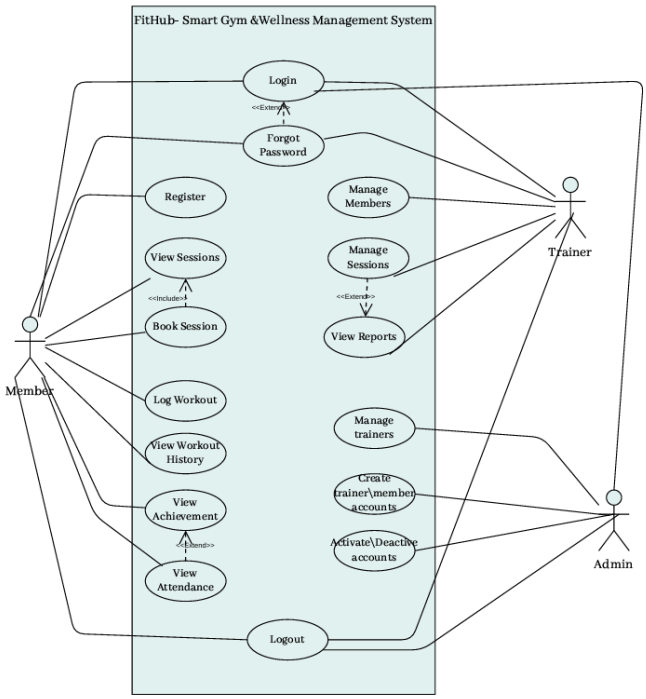


Figure 22: Use Case Diagram

Architecture Diagram

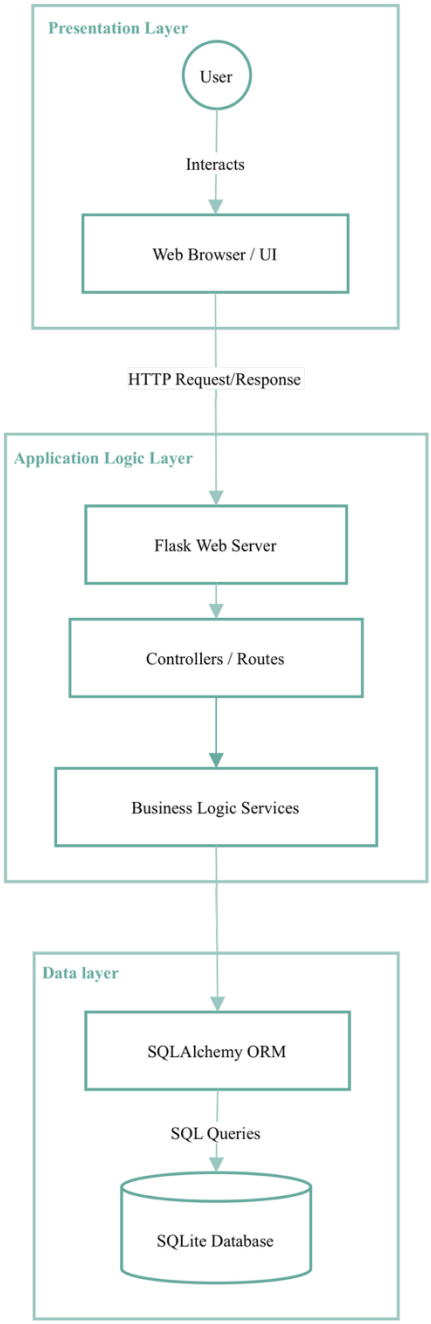


Figure Architecture Diagram

References

1. IEEE Recommended Practice for Software Design Descriptions (IEEE 1016).
2. CSC 305: Software Engineering Course Materials, Term 1, AY 2025/2026 — Dr. Rahma Ahmed.
3. CSC 305 Software Design Description Template (Adapted for Student Projects).
4. Flask Documentation – https://flask.palletsprojects.com/
5. SQLite Documentation – https://www.sqlite.org/docs.html
6. UI Prototype – Created by the project team using Figma